Selenium Java

Interview Questions & API Mocking Guide

Part 1: Scenario-based coding questions

Part 2: Every way to mock APIs in Selenium Java, with code

Part 1: Scenario-Based Coding Questions

Locators and elements

1. A table shows 50 orders over several pages. Write code to find the row with Order ID "ORD-1042" and click its "View" button, handling pagination.
2. Print all links on a page and report any broken ones (status code >= 400).
3. A dropdown is a custom list, not a <select>. Write code to pick "India" from it.
4. An element's ID changes on every page load. Write a stable locator strategy (relative XPath, contains(), CSS) to click the "Submit" button.
5. Find the 3rd item in a dynamic list, and verify the list is sorted alphabetically.

Waits and synchronization

6. A button becomes clickable only after a loader disappears. Write code that waits for the loader to vanish, then clicks.
7. Write a FluentWait that polls every 500ms for up to 20s, ignoring NoSuchElementException.
8. A page loads content through infinite scroll. Scroll until a particular product appears or the list stops growing.
9. Explain why Thread.sleep() is a problem, then rewrite a flaky test to use explicit waits.

Browser handling

10. Clicking a link opens a new tab. Switch to it, verify the title, close it, and return to the parent window.
11. A page has a nested iframe and the login form is inside it. Write code to fill the form and return to the main page.
12. Handle JS alert, confirm and prompt popups: accept one, dismiss one, and type text into the prompt.
13. Upload a file where the input is hidden or a custom button. Then download a file and verify it exists in the download folder.
14. Hover over a menu, then click the sub-menu item (Actions class). Also write drag-and-drop and double-click scenarios.
15. Take a screenshot automatically whenever a test fails (TestNG listener or JUnit extension).

Framework design

16. Write a Page Object Model for a login page and a test that uses it with PageFactory or plain By locators.
17. Make a thread-safe DriverFactory using ThreadLocal<WebDriver> so tests can run in parallel.
18. Read test data from Excel, JSON or CSV and run the same login test with 5 data sets (@DataProvider or @ParameterizedTest).
19. Run the same test on Chrome, Firefox and Edge using a config property or Selenium Grid (RemoteWebDriver).
20. Write a retry mechanism (IRetryAnalyzer) that re-runs failed tests up to 2 times.
21. Write a stale-element-safe click: retry up to 3 times on StaleElementReferenceException.
22. Validate a form: submit it empty, capture all the error messages, and assert each against expected text.

Part 2: Mocking APIs in Selenium Java

Selenium has no built-in mock server. In Selenium 4 you can intercept browser network calls through the Chrome DevTools Protocol (CDP) in Chrome and Edge, use WebDriver BiDi, or put a mock backend or proxy in front of the app.

#	Technique	Browsers	Best for
1	NetworkInterceptor (Selenium 4)	Chrome, Edge	Quick stubs, the easiest option
2	NetworkInterceptor with Filter	Chrome, Edge	Modify real requests or responses
3	CDP Fetch domain	Chrome, Edge	Fine control: fulfill, continue or fail
4	CDP Network (block, throttle, offline)	Chrome, Edge	Slow, offline and blocked-resource cases
5	Capture or verify requests	Chrome, Edge	Asserting the payload the UI sends
6	WebDriver BiDi	Firefox, Chrome	Cross-browser interception
7	WireMock	All	Standalone mock backend
8	MockServer	All	Mock backend with expectations and verification
9	JDK HttpServer / OkHttp MockWebServer	All	No extra dependency
10	BrowserMob Proxy	All	Legacy proxy-based mocking
11	JavaScript injection (fetch / XHR override)	All	Last resort, no tooling needed

Common imports for the CDP examples (match the vNNN package to your Chrome version):

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.*;
import org.openqa.selenium.devtools.*;
import org.openqa.selenium.remote.http.*;
import java.time.Duration;
import java.util.*;
```

1. NetworkInterceptor: stub a response

Question: the product page calls GET /api/products. Mock it with 1 product and verify the UI. Change setStatus(...) to mock 401, 404 or 500.

```
String json = "[{\"id\":1,\"name\":\"Mock Laptop\"}]";

try (NetworkInterceptor ni = new NetworkInterceptor(driver,
        Route.matching(req -> req.getMethod() == HttpMethod.GET
                          && req.getUri().contains("/api/products"))
             .to(() -> req -> new HttpResponse()
                 .setStatus(200)
                 .addHeader("Content-Type", "application/json")
                 .setContent(Contents.utf8String(json))))) {

    driver.get("https://your-app.com/products");
    int cards = driver.findElements(By.cssSelector(".product-card")).size();
    Assert.assertEquals(cards, 1);
}
```

1b. Simulate a server error (500) and verify the error UI

```
@Test
public void shouldShowErrorWhenApiFails() {
    try (NetworkInterceptor ni = new NetworkInterceptor(driver,
            Route.matching(req -> req.getUri().contains("/api/orders"))
                 .to(() -> req -> new HttpResponse()
                     .setStatus(500)
                     .addHeader("Content-Type", "application/json")
                     .setContent(Contents.utf8String("{\"error\":\"Internal Server Error\"}"))))) {

        driver.get("https://your-app.com/orders");
```

```
        String msg = new WebDriverWait(driver, Duration.ofSeconds(10))
            .until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".error-banner")))
            .getText();
        Assert.assertEquals(msg, "Something went wrong");
    }
}
```

2. NetworkInterceptor with Filter: modify the real response

The real call goes through and you tamper with the result. Useful for changing one field without rewriting the whole payload.

```
Filter modifyResponse = next -> req -> {
    HttpResponse res = next.execute(req);                // real call
    if (req.getUri().contains("/api/user")) {
        String body = Contents.string(res)
                .replace("\"role\":\"user\"", "\"role\":\"admin\"");
        return new HttpResponse()
                .setStatus(res.getStatus())
                .addHeader("Content-Type", "application/json")
                .setContent(Contents.utf8String(body));
    }
    return res;
};

try (NetworkInterceptor ni = new NetworkInterceptor(driver, modifyResponse)) {
    driver.get("https://your-app.com/profile");
}
```

Add Thread.sleep(3000) before returning to simulate a slow API.

3. CDP Fetch domain: fulfill, continue, fail

Mock only the calls you care about, fail some, and let the rest pass through.

```
import org.openqa.selenium.devtools.v127.fetch.Fetch;
import org.openqa.selenium.devtools.v127.fetch.model.HeaderEntry;
import org.openqa.selenium.devtools.v127.network.model.ErrorReason;

DevTools dt = ((HasDevTools) driver).getDevTools();
dt.createSession();
dt.send(Fetch.enable(Optional.empty(), Optional.empty()));

String body = Base64.getEncoder().encodeToString("{\"status\":\"mocked\"}".getBytes());

dt.addListener(Fetch.requestPaused(), req -> {
    String url = req.getRequest().getUrl();

    if (url.contains("/api/status")) {                       // MOCK
        dt.send(Fetch.fulfillRequest(req.getRequestId(), 200,
            Optional.of(List.of(new HeaderEntry("Content-Type", "application/json"))),
            Optional.empty(), Optional.of(body), Optional.empty()));

    } else if (url.contains("/api/analytics")) {             // FAIL (network error)
        dt.send(Fetch.failRequest(req.getRequestId(), ErrorReason.FAILED));

    } else {                                                 // PASS THROUGH
        dt.send(Fetch.continueRequest(req.getRequestId(), Optional.empty(),
            Optional.empty(), Optional.empty(), Optional.empty(), Optional.empty()));
    }
});

driver.get("https://your-app.com");
```

The vNNN package and the argument counts depend on your Selenium and Chrome versions. continueRequest also lets you rewrite the URL, method, headers and POST data.

4. CDP Network: block, throttle, offline

Block URLs (ads, analytics, images):

```
import org.openqa.selenium.devtools.v127.network.Network;

DevTools dt = ((HasDevTools) driver).getDevTools();
dt.createSession();
dt.send(Network.enable(Optional.empty(), Optional.empty(), Optional.empty()));
```

```
dt.send(Network.setBlockedURLs(List.of("*.png", "*google-analytics*", "*/ads/*")));
```

Slow network or offline mode (more stable across versions):

```
import org.openqa.selenium.chromium.ChromiumDriver;
import org.openqa.selenium.chromium.ChromiumNetworkConditions;

ChromiumNetworkConditions nc = new ChromiumNetworkConditions();
nc.setOffline(false);
nc.setLatency(Duration.ofMillis(2000));
nc.setDownloadThroughput(50 * 1024);   // bytes/sec
nc.setUploadThroughput(20 * 1024);
((ChromiumDriver) driver).setNetworkConditions(nc);

// offline test
nc.setOffline(true);
((ChromiumDriver) driver).setNetworkConditions(nc);
```

Add custom headers to every request:

```
dt.send(Network.setExtraHTTPHeaders(new Headers(Map.of("X-Test-Mode", "true"))));
```

5. Capture and verify requests and responses

Question: when the user clicks Place Order, verify the app sends the right JSON payload.

```
List<String> postBodies = new ArrayList<>();
Map<String, Integer> statuses = new HashMap<>();

dt.addListener(Network.requestWillBeSent(), e -> {
    if (e.getRequest().getUrl().contains("/api/orders")
            && "POST".equals(e.getRequest().getMethod())) {
        e.getRequest().getPostData().ifPresent(postBodies::add);
    }
});

dt.addListener(Network.responseReceived(), e ->
    statuses.put(e.getResponse().getUrl(), e.getResponse().getStatus()));

driver.findElement(By.id("place-order")).click();
Assert.assertTrue(postBodies.get(0).contains("\"quantity\":2"));
```

6. WebDriver BiDi: cross-browser interception

Works in Firefox and Chrome and needs the webSocketUrl capability. The BiDi Network API has changed between Selenium 4.x releases, so check your version's Javadoc.

```
import org.openqa.selenium.bidi.module.Network;
import org.openqa.selenium.bidi.network.*;

FirefoxOptions options = new FirefoxOptions();
options.setCapability("webSocketUrl", true);
WebDriver driver = new FirefoxDriver(options);

try (Network network = new Network(driver)) {
    network.addIntercept(new AddInterceptParameters(InterceptPhase.BEFORE_REQUEST_SENT));

    network.onBeforeRequestSent(event -> {
        String url = event.getRequest().getUrl();
        if (url.contains("/api/products")) {
            // block / fail / modify the request here (API depends on version)
        }
        network.continueWithoutModifications(event.getRequest().getRequestId());
    });

    driver.get("https://your-app.com/products");
}
```

If your Selenium version lacks full response mocking over BiDi, use WireMock or a proxy for Firefox.

7. WireMock: standalone mock backend

Maven dependency: org.wiremock:wiremock-standalone.

```
import com.github.tomakehurst.wiremock.WireMockServer;
import static com.github.tomakehurst.wiremock.client.WireMock.*;
```

```
WireMockServer wm = new WireMockServer(8089);
wm.start();

wm.stubFor(get(urlEqualTo("/api/products"))
    .willReturn(aResponse().withStatus(200)
        .withHeader("Content-Type", "application/json")
        .withHeader("Access-Control-Allow-Origin", "*")   // CORS if app is on another origin
        .withBody("[{\"id\":1,\"name\":\"Stub Item\"}]")));

// error + delay + conditional stubs
wm.stubFor(get(urlEqualTo("/api/orders"))
    .willReturn(aResponse().withStatus(500).withFixedDelay(2000)));

wm.stubFor(post(urlEqualTo("/api/login"))
    .withRequestBody(matchingJsonPath("$.username", equalTo("admin")))
    .willReturn(okJson("{\"token\":\"abc123\"}")));

driver.get("http://localhost:3000");   // app configured with API_URL=http://localhost:8089

wm.verify(1, getRequestedFor(urlEqualTo("/api/products")));
wm.stop();
```

8. MockServer

Maven dependency: org.mock-server:mockserver-netty.

```
import org.mockserver.integration.ClientAndServer;
import static org.mockserver.model.HttpRequest.request;
import static org.mockserver.model.HttpResponse.response;

ClientAndServer ms = ClientAndServer.startClientAndServer(1080);

ms.when(request().withMethod("GET").withPath("/api/products"))
  .respond(response().withStatusCode(200)
      .withHeader("Content-Type", "application/json")
      .withHeader("Access-Control-Allow-Origin", "*")
      .withBody("[{\"id\":1,\"name\":\"MockServer Item\"}]"));

driver.get("http://localhost:3000");

ms.verify(request().withPath("/api/products"));
ms.stop();
```

9. Plain JDK HttpServer (zero dependencies)

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;

HttpServer server = HttpServer.create(new InetSocketAddress(8090), 0);
server.createContext("/api/products", ex -> {
    byte[] data = "[{\"id\":1,\"name\":\"JDK Mock\"}]".getBytes();
    ex.getResponseHeaders().add("Content-Type", "application/json");
    ex.getResponseHeaders().add("Access-Control-Allow-Origin", "*");
    ex.sendResponseHeaders(200, data.length);
    ex.getResponseBody().write(data);
    ex.close();
});
server.start();
// ... run test ...
server.stop(0);
```

With OkHttp MockWebServer you can queue responses with server.enqueue(new MockResponse().setBody(...)) and inspect them with server.takeRequest().

10. BrowserMob Proxy (legacy, unmaintained)

```
import net.lightbody.bmp.BrowserMobProxy;
import net.lightbody.bmp.BrowserMobProxyServer;
import net.lightbody.bmp.client.ClientUtil;
import io.netty.buffer.Unpooled;
import io.netty.handler.codec.http.*;
import java.nio.charset.StandardCharsets;

BrowserMobProxy proxy = new BrowserMobProxyServer();
proxy.start(0);

proxy.addRequestFilter((request, contents, info) -> {
```

```
    if (info.getOriginalUrl().contains("/api/products")) {
        FullHttpResponse r = new DefaultFullHttpResponse(HttpVersion.HTTP_1_1,
            HttpResponseStatus.OK,
            Unpooled.copiedBuffer("[{\"id\":1}]", StandardCharsets.UTF_8));
        r.headers().add("Content-Type", "application/json");
        return r;
    }
    return null;   // pass through
});

ChromeOptions options = new ChromeOptions();
options.setProxy(ClientUtil.createSeleniumProxy(proxy));
options.setAcceptInsecureCerts(true);
WebDriver driver = new ChromeDriver(options);
```

11. JavaScript injection (works in any browser)

Override fetch after page load. It only affects calls made after injection, so trigger the action after you inject.

```
((JavascriptExecutor) driver).executeScript(
    "const realFetch = window.fetch;" +
    "window.fetch = (url, opts) => {" +
    "  if (String(url).includes('/api/products')) {" +
    "    return Promise.resolve(new Response(" +
    "      JSON.stringify([{id:1, name:'JS Mock'}])," +
    "      {status: 200, headers: {'Content-Type': 'application/json'}}));" +
    "  }" +
    "  return realFetch(url, opts);" +
    "};");
driver.findElement(By.id("load-products")).click();
```

To inject before the page's own scripts run in Chrome, use CDP:

```
((ChromeDriver) driver).executeCdpCommand("Page.addScriptToEvaluateOnNewDocument",
    Map.of("source", "window.fetch = () => Promise.resolve(new Response('[]'));"));
```

Selenium Grid / RemoteWebDriver

CDP features need an augmented driver:

```
WebDriver remote = new RemoteWebDriver(new URL("http://grid:4444"), new ChromeOptions());
WebDriver driver = new Augmenter().augment(remote);
```

Which one to use in an interview answer

- **Chrome-only UI test, quick stub:** NetworkInterceptor
- **Fine-grained control, or simulating network failures:** CDP Fetch and Network
- **Firefox or cross-browser:** BiDi, or WireMock and MockServer
- **Reusable mock backend shared by many tests:** WireMock
- **No tooling allowed:** JS injection

Follow-up questions interviewers ask

1. Why mock APIs in UI tests?
Speed, stability, testing error and edge cases, no backend dependency.

2. What are the limitations of CDP-based mocking?
Chromium-only; version-matched DevTools packages; with Grid/RemoteWebDriver you must use new Augmenter().augment(driver).

3. How would you do this in Firefox?
WireMock, a proxy like BrowserMob or MockServer, or WebDriver BiDi support in Selenium 4.

4. What is the difference between mocking at the network layer and stubbing the backend?
Network-layer mocking changes what the browser receives and needs no app changes; a stubbed backend replaces the server the app talks to and needs the API URL to be configurable.

5. How do you simulate slow responses or timeouts?

A delay inside the handler or filter, WireMock withFixedDelay, or Network.emulateNetworkConditions.

6. How do you block images, ads or analytics calls to speed up tests?

Network.setBlockedURLs.

7. When should you not mock?

End-to-end smoke tests and contract validation, where real integration matters.